

CLAUDE CODE DEEP DIVE

CLAUDE.md

Source: CLAUDE.md

Guidance for Claude Code when working in this repository.

■ Project

A minimal **full-stack todo app** used as a Claude Code learning sandbox (Tutorial 5). It demonstrates wiring a Vite + React + TypeScript frontend to an Express + TypeScript backend, with TanStack Query v5 handling fetching, caching, and mutations. Storage is an in-memory array on the server — todos reset on restart.

See [README.md](#) for the full feature walk-through and [Notes.md](#) for the tutorial steps that produced this project (resume, plan mode, MCP, hooks, subagents).

■ Layout

```
6.Tutorial5/
├─ README.md           # Full project docs (tech stack, run, API)
├─ Notes.md            # Tutorial walkthrough notes
├─ MULTI_AGENT_GUIDE.md # Subagent reference doc (paste into prompts)
├─ CLAUDE.md           # This file
├─ .claude/
│   └─ settings.local.json # Project permissions + hooks
│       └─ hooks/          # on-task-done / on-task-pending / on-needs-approval
├─ logs/               # Hook output logs
├─ client/              # Vite + React 19 + TS + TanStack Query v5
│   └─ src/
│       ├── main.tsx      # QueryClientProvider wraps <App />
│       ├── App.tsx       # useQuery + 3x useMutation (add/toggle/delete)
│       ├── api.ts        # Typed fetch helpers
│       └─ App.css
├─ server/              # Express + TS + tsx
│   └─ src/
│       ├── index.ts      # cors + express.json + 4 CRUD routes
│       ├── store.ts      # In-memory todos array (crypto.randomUUID ids)
│       └─ types.ts       # Todo type
```

■ How to run

Two terminals from the project root:

BASH

```
# terminal 1 – backend on :3001 (auto-reload via tsx watch)
cd server && npm run dev

# terminal 2 – frontend on :5173 (Vite HMR)
cd client && npm run dev
```

Open <http://localhost:5173> to verify the UI; the backend exposes REST at <http://localhost:3001/todos>.

■ Common scripts

Where	Command	Purpose
server	npm run dev	Run dev server with tsx watch
server	npm run typecheck	tsc --noEmit
server	npm run build	Compile TS → dist/
server	npm start	Run compiled dist/index.js
client	npm run dev	Vite dev server
client	npm run build	tsc -b && vite build
client	npm run lint	ESLint
client	npm run preview	Serve production build

■ API contract (server)

```
GET    /todos      → Todo[]
POST   /todos      { title }      → Todo (201)
PATCH /todos/:id  { title?, completed? } → Todo (404 if missing)
DELETE /todos/:id  → 204 (404 if missing)

Todo = { id: string (uuid), title: string, completed: boolean }
```

■ Conventions

- **TypeScript everywhere.** Both client/ and server/ are strict-TS projects.
- **Server is ESM** ("type": "module" in server/package.json). Use import syntax; no CommonJS.
- **State in client/ flows through TanStack Query.** Don't hand-roll useEffect + fetch for new endpoints — add a typed helper in client/src/api.ts and a useQuery / useMutation in the

component. Mutations should call `queryClient.invalidateQueries({ queryKey: ['todos'] })` on success.

- **Server state lives in** `server/src/store.ts`. Treat it as the single source of truth — routes in `index.ts` call `store.list/create/update/remove`, they don't touch the array directly.
- **Ports are hardcoded:** server `3001`, client `5173`. If you change the server port, also update the `API` constant in `client/src/api.ts`.
- **No persistence.** Don't add a database without first asking — restarting the server is *expected* to clear todos in this tutorial.

■ Claude Code workflow notes

- The project has **hooks configured** in `.claude/settings.local.json` for `Stop`, `PreToolUse`, and `Notification (permission_prompt)`. They write to `logs/`. Don't disable or rewrite them without asking — they're tutorial artefacts.
- A **Context7 MCP server** was used during scaffolding to fetch latest docs for Vite and TanStack Query. Continue to prefer Context7 over web search for library documentation.
- For exploring the codebase, prefer the **Explore** subagent (read-only). For multi-step changes, use **general-purpose**. See `MULTI_AGENT_GUIDE.md` for the full subagent playbook.
- When verifying a UI change, actually load <http://localhost:5173> in a browser and exercise add / toggle / delete — `tsc` and `vite build` only prove the code compiles, not that the feature works.

■ Things to avoid

- Adding new top-level dependencies (DB, ORM, state library) — this is a deliberately tiny sandbox.
- Splitting `store.ts` into many files; the in-memory store is supposed to fit on one screen.
- Touching `node_modules/` or `package-lock.json` manually — use `npm install` from the right subdirectory.
- Skipping hooks (`--no-verify`) or bypassing permissions in `.claude/settings.local.json`.